Interview Study Guide — Research & Report Agent

For Freshers | Zero to Interview Ready

PART 1: WHAT DID WE BUILD?

We built a **full-stack Agentic AI application**.

Think of it like this:

> You type a topic → A robot goes to the internet, reads articles, understands them, and writes a professional report for you — automatically.

The Simple Flow

```
You type "Agentic AI trends 2025"
        ↓
Step 1: Agent searches Google (using Tavily)
        ↓
Step 2: AI reads the search results and understands them (using Groq/LLM)
        ↓
Step 3: AI writes a full report (Executive Summary, Key Findings, etc.)
        ↓
Report appears on screen + can be downloaded
```

PART 2: EVERY TECHNOLOGY EXPLAINED (SIMPLY)

---

1. Python

What it is: A programming language. Like English for computers. **Why we used it:** All AI/ML tools are built for Python. It's the #1 language for AI. **Interview Question:** *"Why is Python used in AI?"* **Answer:** "Python has the most AI/ML libraries like LangChain, TensorFlow, and PyTorch. It's simple to write and has a massive community."

---

2. LangGraph

What it is: A tool that helps you build AI "workflows" with multiple steps. **Simple analogy:** Like a flowchart. Each box in the flowchart is a "node". The arrows between boxes are "edges". **Why we used it:** Our agent has 3 steps (Search → Analyze → Write). LangGraph connects these steps and passes data between them.

Key Concepts:

- **Node** = One step in the agent (e.g., the Search step)
- **Edge** = Connection between steps (Search → Analyze)

- **State** = The data passed between steps (topic, results, report)
- **Graph** = The full pipeline of nodes + edges

Interview Question: *"What is LangGraph?"* **Answer:** "LangGraph is a library built on LangChain for building stateful, multi-step AI agents as graphs. Each node is a function, and edges define the flow between them. It gives more control over agent behavior compared to simple chains."

---

3. LLM (Large Language Model)

What it is: The AI brain. It understands and generates human language. **Simple analogy:** ChatGPT is an LLM. It reads text and writes text back. **Examples:** GPT-4 (OpenAI), Claude (Anthropic), Llama (Meta), Gemini (Google) **We used:** Llama 3.3 70B from Groq

Key Concepts:

- **Prompt** = The instruction/question you give to the LLM
- **Response/Completion** = What the LLM writes back
- **Token** = A piece of a word. "Hello" = 1 token. LLMs charge by tokens.
- **Context Window** = How much text the LLM can read at once

Interview Question: *"What is an LLM?"* **Answer:** "A Large Language Model is a deep learning model trained on massive amounts of text data. It can understand and generate human language. Examples include GPT-4, Claude, and Llama. They work by predicting the next token based on the input."

---

4. Groq

What it is: A company that runs LLMs very fast (using special hardware called LPU). **Why we used it:** It's free, fast, and gives access to powerful models like Llama 3.3 70B. **Simple analogy:** Groq is like a very fast car engine. The LLM (Llama) is the car. Groq just makes it run faster.

Interview Question: *"Why did you use Groq instead of OpenAI?"* **Answer:** "Groq provides a free API with very fast inference using their LPU hardware. For a portfolio project, it was ideal because it's free, has generous rate limits, and supports powerful open-source models like Llama 3.3 70B."

---

5. Tavily API

What it is: A search engine built specifically for AI agents. **Simple analogy:** Like Google Search, but instead of showing you web pages, it gives clean text that AI can read. **Why we used it:** Normal Google search gives HTML (messy). Tavily gives clean text perfect for LLMs.

Interview Question: *"How does your agent search the web?"* **Answer:** "We use the Tavily API, which is a search engine designed for AI agents. It returns clean, structured text from web pages instead of raw HTML, making it easy for the LLM to process the results."

---

6. FastAPI

What it is: A Python tool to build APIs (backend servers). **Simple analogy:** A waiter in a restaurant. Frontend says "I want a report on X". FastAPI takes that order, gives it to the agent, and brings back the report. **Why we used it:** It's fast, modern, and automatically creates documentation at `/docs`.

Key Concepts:

- **API** = A way for two programs to talk to each other
- **Endpoint** = A URL that does something. We have `/research`
- **POST request** = Sending data to the server (we send the topic)
- **JSON** = The format data is sent in. Like: `{"topic": "AI trends"}`
- **CORS** = Security rule that controls which websites can talk to your API

Interview Question: *"What is an API?"* **Answer:** "An API (Application Programming Interface) is a way for different software systems to communicate. In our project, the React frontend sends a POST request with the research topic to our FastAPI backend, which runs the agent and returns the report as JSON."

---

7. React

What it is: A JavaScript tool for building web interfaces (what users see). **Simple analogy:** React builds the "face" of the app — the input box, the button, the report display. **Why we used it:** Most companies use React for frontends. It's the most in-demand frontend skill.

Key Concepts:

- **Component** = A reusable piece of UI (like a button or input box)
- **State** = Data that can change (like the report text appearing after generation)
- **useState** = A React tool to manage changing data
- **Props** = Data passed between components
- **JSX** = HTML written inside JavaScript

Interview Question: *"What is React?"* **Answer:** "React is a JavaScript library for building user interfaces. It uses a component-based architecture where each UI element is a reusable component. We used React to build the frontend where users enter topics and view generated reports."

---

8. Vite

What it is: A tool that runs and builds React apps super fast. **Simple analogy:** Like a compiler/runner for React. You don't need to understand it deeply. **Why we used it:** It's faster than the old Create React App tool.

---

9. REST API

What it is: A standard way of building APIs using HTTP methods. **Key methods:**

- **GET** = Read data (like loading a webpage)
- **POST** = Send data (like submitting a form — we use this)
- **PUT** = Update data
- **DELETE** = Delete data

Interview Question: *"What HTTP method does your agent use?"* **Answer:** "We use a POST request because we're sending data (the research topic) to the backend. GET requests are for retrieving data without a body, while POST allows us to send a JSON body."

---

10. Agentic AI

What it is: AI that can take actions, make decisions, and complete multi-step tasks on its own — without a human guiding every step. **Difference from ChatGPT:** ChatGPT just responds to one message. An agent can search the web, write code, send emails, and make decisions over multiple steps.

Key Concepts:

- **Agent** = AI that takes actions autonomously
- **Tool** = Something the agent can use (like web search, calculator, database)
- **Tool Use / Function Calling** = When the LLM decides to use a tool
- **ReAct Pattern** = Agent Reasons then Acts, repeatedly until done
- **Multi-agent** = Multiple AI agents working together

Interview Question: *"What is Agentic AI?"* **Answer:** "Agentic AI refers to AI systems that can autonomously plan and execute multi-step tasks. Unlike a simple chatbot that responds to one message, an agent can use tools like web search, decide what actions to take, and iterate until it completes a goal. Our project implements this with LangGraph, where each node represents an autonomous step."

PART 3: HOW OUR CODE WORKS (LINE BY LINE CONCEPT)

The Agent (agent.py)

```
# 1. State — the shared memory between all nodes
class ResearchState(TypedDict):
    topic: str           # What to research
    search_results: list # Web search results
    analysis: str        # AI's understanding
    report: str          # Final report

# 2. Node 1 — Search
def search_node(state):
    results = tavily.search(state["topic"])  # Search the web
    state["search_results"] = results
    return state

# 3. Node 2 — Analyze
def analyze_node(state):
    response = llm.invoke([...])  # Ask LLM to read results
    state["analysis"] = response.content
    return state

# 4. Node 3 — Write Report
def write_report_node(state):
    response = llm.invoke([...])  # Ask LLM to write report
```

```
    state["report"] = response.content
    return state

# 5. Graph — connect the nodes
graph = StateGraph(ResearchState)
graph.add_node("search", search_node)
graph.add_node("analyze", analyze_node)
graph.add_node("write_report", write_report_node)
graph.add_edge("search", "analyze")       # search runs first
graph.add_edge("analyze", "write_report") # then analyze
# then write_report → END
```

The Backend (main.py)

```
@app.post("/research")           # When frontend calls /research
async def research(request):
    report = run_research_agent(request.topic)  # Run the agent
    return {"report": report}    # Send report back
```

The Frontend (App.jsx)

```
// When user clicks "Generate Report":
const res = await fetch("https://backend-url/research", {
    method: "POST",
    body: JSON.stringify({ topic: "AI trends" })  // Send topic
})
const data = await res.json()  // Get report back
setReport(data.report)         // Show on screen
```

PART 4: MOST ASKED INTERVIEW QUESTIONS

Q1: "Explain your project in 2 minutes"

Answer: "I built a Research & Report Agent — a full-stack AI application. The user enters any research topic, and the system automatically searches the web, analyzes the results using an LLM, and generates a structured professional report. I used LangGraph to orchestrate a 3-node agent pipeline: a search node using Tavily API, an analysis node, and a report-writing node — both powered by Llama 3.3 70B on Groq. The backend is built with FastAPI and the frontend with React. It's fully deployed — backend on Render, frontend on Vercel."

Q2: "What is the difference between a chatbot and an agent?"

Answer: "A chatbot responds to a single message with a single response. An agent can plan, take multiple actions, use external tools, and complete complex multi-step goals autonomously. For example, our agent

doesn't just answer 'what is AI' — it searches the web, reads 5 sources, synthesizes them, and writes a full report."

Q3: "What is RAG?"

Answer: "RAG stands for Retrieval Augmented Generation. Instead of relying on the LLM's training data alone, RAG first retrieves relevant documents (from a database or the web), then feeds them to the LLM to generate a more accurate, up-to-date answer. Our project uses a form of RAG — we retrieve web pages via Tavily, then feed them to the LLM."

Q4: "What is a vector database?"

Answer: "A vector database stores data as numerical vectors (embeddings). When you want to find similar content, it compares vectors mathematically. It's used in RAG systems to find relevant documents. Examples: Pinecone, ChromaDB, Weaviate."

Q5: "What is prompt engineering?"

Answer: "Prompt engineering is the practice of designing effective instructions for LLMs to get better outputs. In our project, we carefully wrote system prompts like 'You are a research analyst. Extract key insights...' to guide the LLM to produce structured, useful outputs."

Q6: "What is LangChain?"

Answer: "LangChain is a framework for building LLM-powered applications. It provides tools for chaining LLM calls, connecting to external data sources, and building agents. LangGraph is built on top of LangChain and adds support for stateful, graph-based agent workflows."

Q7: "Why did you use LangGraph instead of a simple loop?"

Answer: "LangGraph provides a clear, maintainable structure for multi-step agents. It manages state automatically between nodes, supports conditional branching, and makes it easy to add new steps. A simple loop would be harder to scale, debug, and extend."

PART 5: CONCEPTS TO STUDY NEXT

Topic	Why Important	Resource
RAG (Retrieval Augmented Generation)	Most common enterprise AI use case	LangChain docs
Vector Databases	Used in RAG systems	ChromaDB docs
Prompt Engineering	Makes LLM outputs better	OpenAI cookbook
LangChain basics	Foundation for LangGraph	LangChain docs

Topic	Why Important	Resource
REST APIs	Used everywhere	freeCodeCamp
React hooks (useState, useEffect)	Frontend interviews	React docs
Git & GitHub	Used in every job	GitHub guides

PART 6: YOUR TECH STACK SUMMARY

```
┌──────────────────────────────────────────┐
│          RESEARCH & REPORT AGENT         │
├──────────────────────────────────────────┤
│  FRONTEND         │  React + Vite        │
│  Deployed on      │  Vercel              │
├──────────────────────────────────────────┤
│  BACKEND          │  FastAPI (Python)     │
│  Deployed on      │  Render              │
├──────────────────────────────────────────┤
│  AI AGENT         │  LangGraph           │
│  LLM              │  Llama 3.3 70B (Groq)│
│  Web Search       │  Tavily API          │
├──────────────────────────────────────────┤
│  LIVE DEMO        │  research-report-agent-│
│                   │  gules.vercel.app    │
│  GITHUB           │  github.com/tanmay5654/│
│                   │  research-report-agent│
└──────────────────────────────────────────┘
```

Study this guide before every interview. You built this — you understand it.